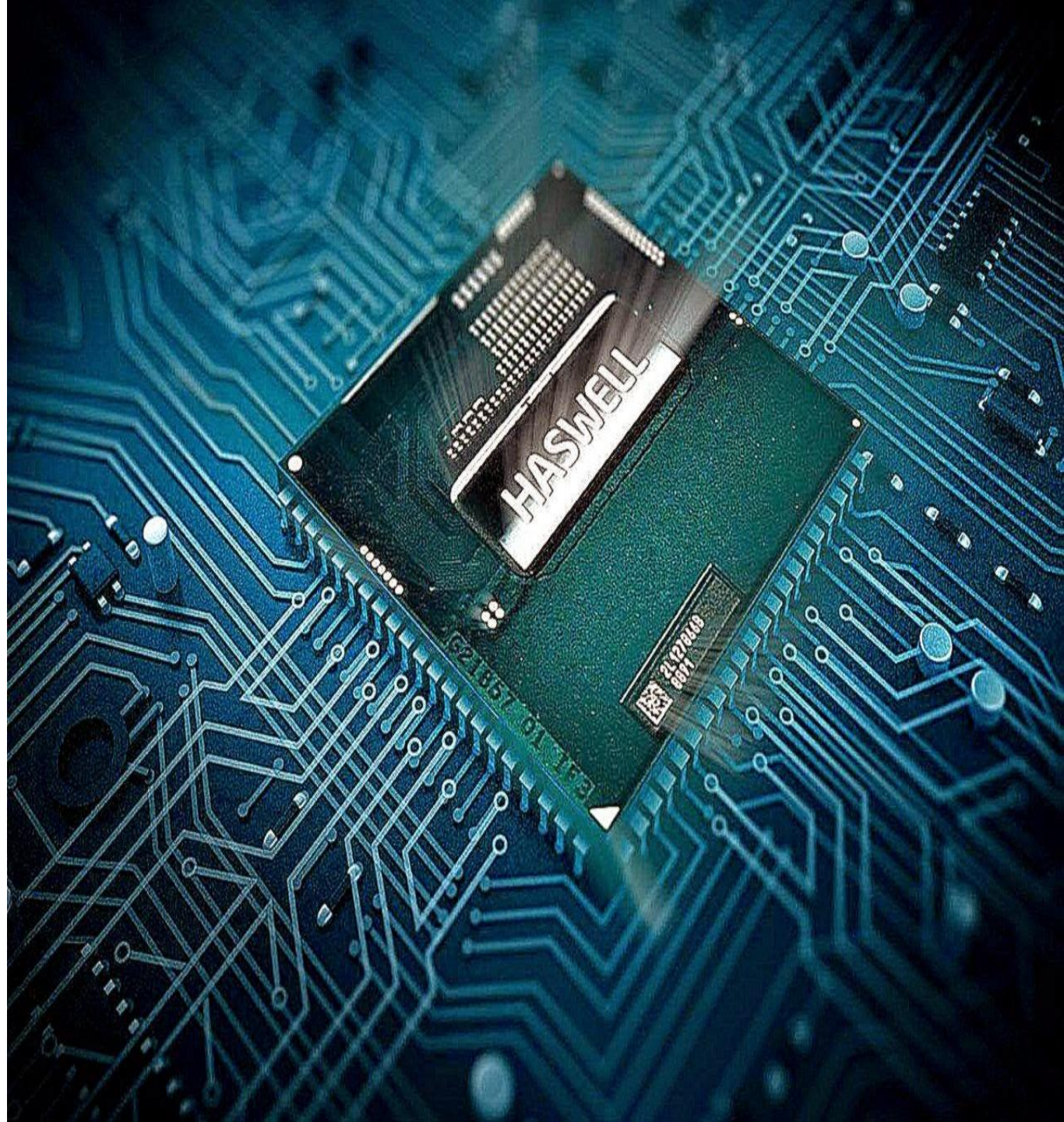
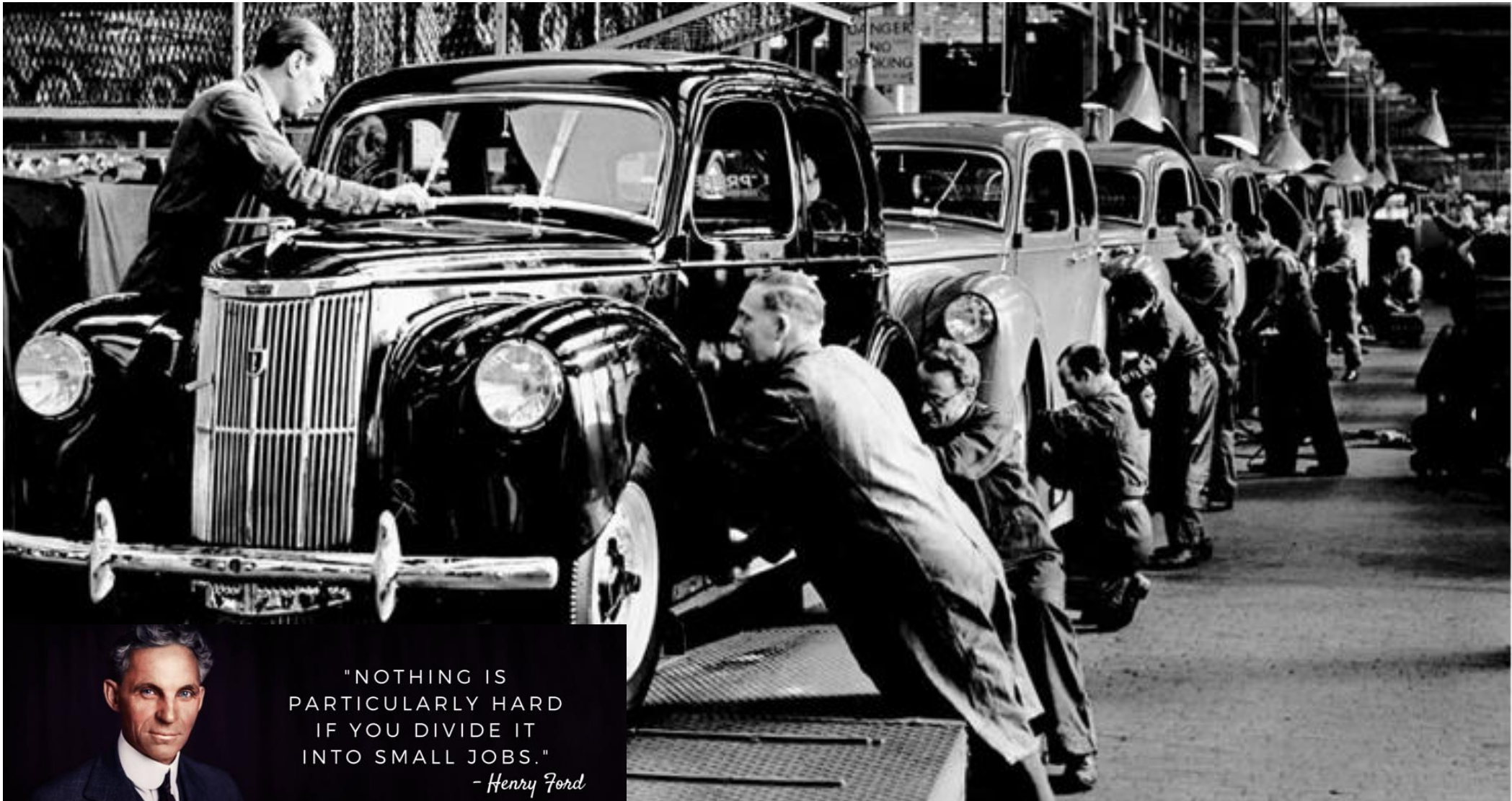


Today's Lecture

- MIPS Processor Design
 - Basic elements of processor
 - Datapath and Control paths
- Processor Design Modes
 - Single-cycle Design (COAHP 4.4)
 - Multi-cycle Design
 - **Pipelined Design** (COAHP 4.5,4.6)
- Reading Materials:
 - Chapter 4 – COAHP (4.5,4.6)
 - Chapters 14,15 – COAWS



HENRY FORD'S BIG IDEA: ASSEMBLY LINE (PIPELINED ASSEMBLY)



PIPELINING INSTRUCTION EXECUTION

Similar to the use of an
assembly line in a
manufacturing plant/Laundry



New inputs are accepted at one
end before previously accepted
inputs appear as outputs at the
other end

To apply this concept to instruction execution:

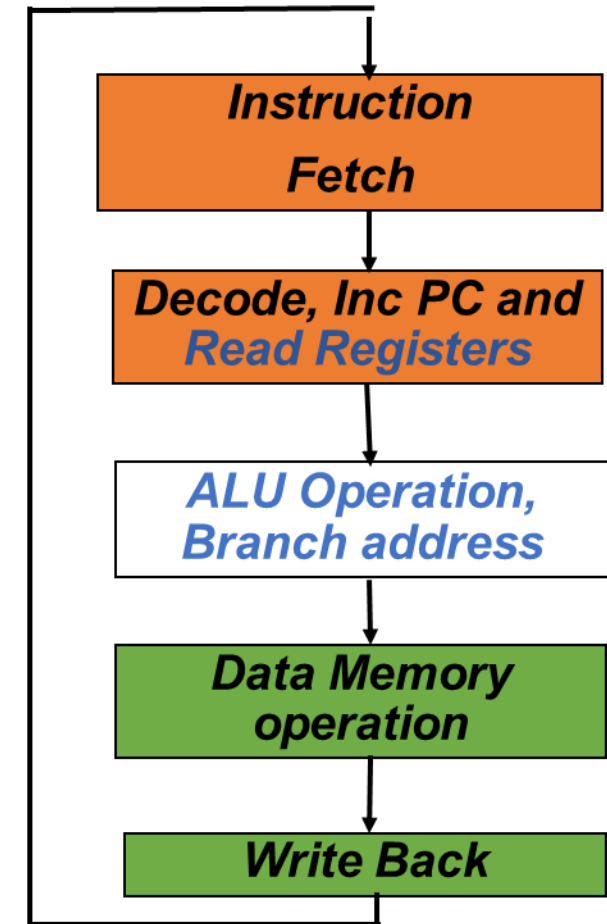
We must recognize that an instruction has a number of stages.

Stage_(i) is the Producer and Stage_(i+1) is the Consumer

STAGES OF EXECUTION IN PIPELINED MIPS

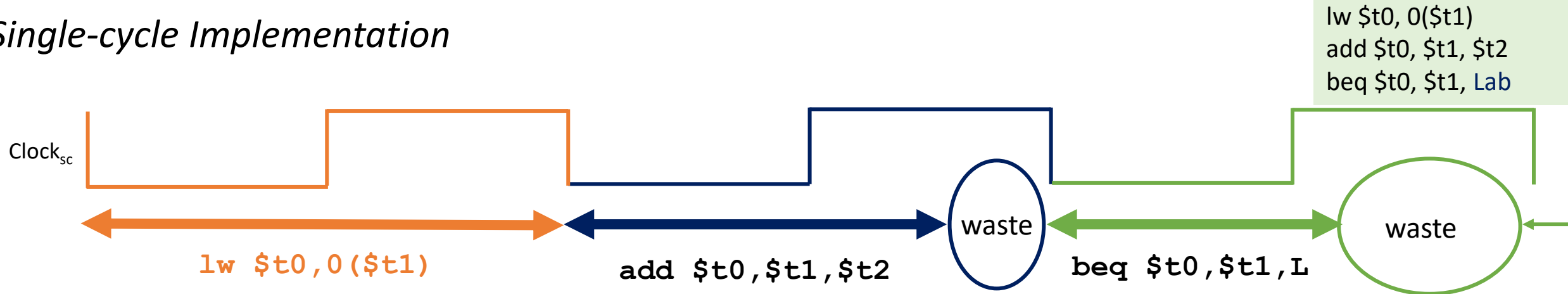
5 stage instruction pipeline

- 1) I-fetch: Fetch Instruction, Increment PC
- 2) Decode: Instruction, Read Registers
- 3) Execute:
Mem-reference: Calculate Address
R-format: Perform ALU Operation
- 4) Memory:
Load: Read Data from Data Memory
Store: Write Data to Data Memory
- 5) Write Back: Write Data to Register



SINGLE-CYCLE VS. MULTICYCLE Vs PIPELINED PROCESSOR

Single-cycle Implementation

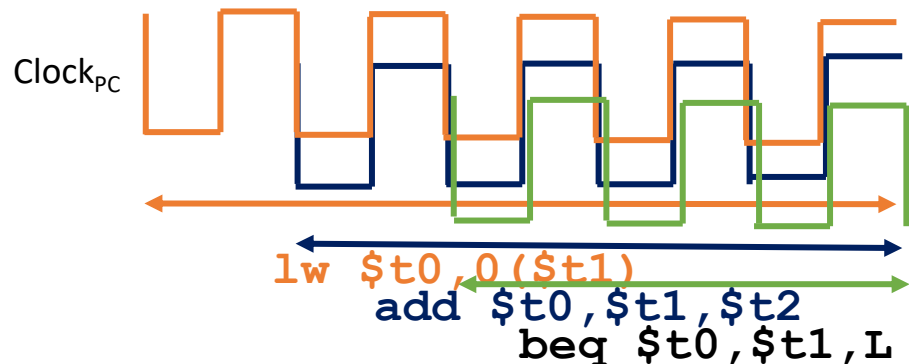


Multicycle Implementation



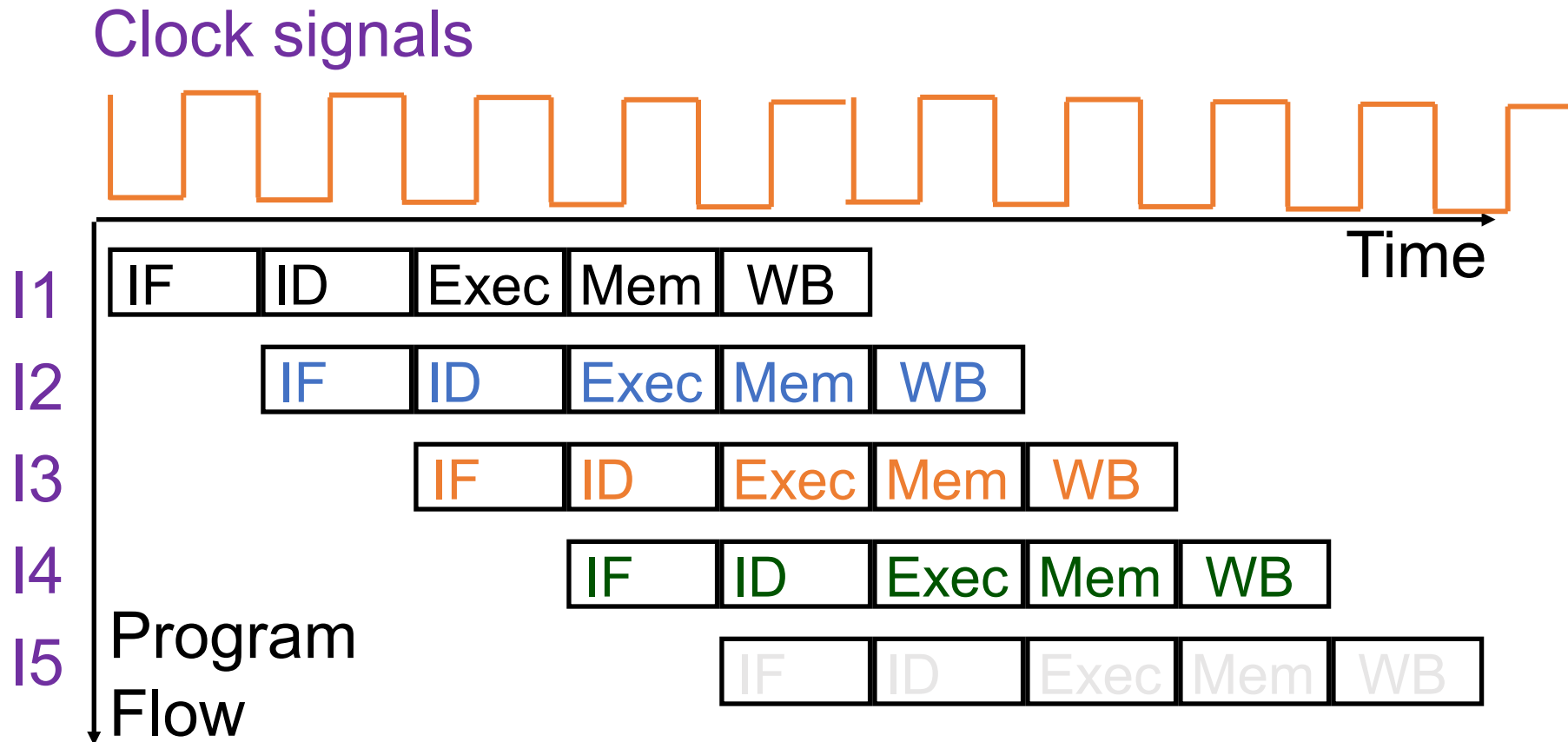
Multicycle Implementation
→ less waste & higher performance (*most often*)

Pipelined Implementation



Pipelined Implementation → Even Higher performance than multi-cycle

PIPELINE DIAGRAM



- To simplify pipeline, every instruction takes same number of steps, called [stages](#)
- One clock cycle per stage

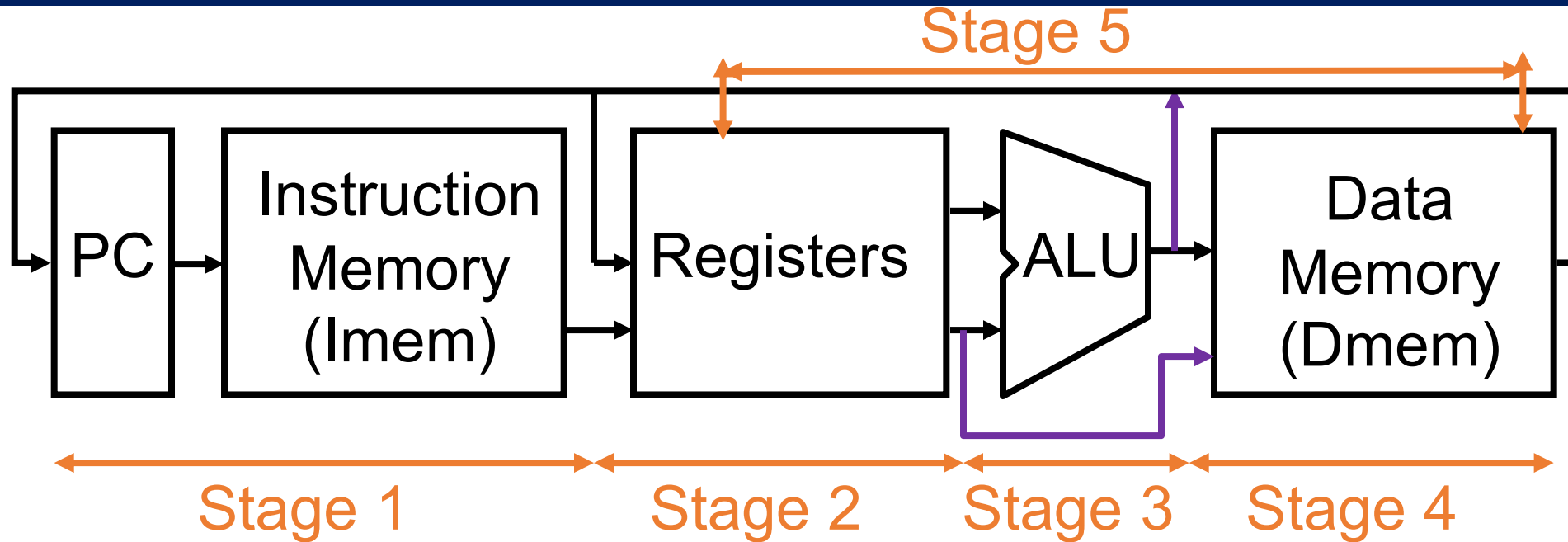
ADVANTAGE OF PIPELINING

- We keep all parts of the data path, busy all the time
- Let us assume that all the 5 stages do the same amount of **work**
 - **Without** pipelining, every **T** seconds, an instruction completes its **execution**
 - **With** pipelining, every **T/5** seconds, a new instruction completes its **execution**
 - Throughput vs. Latency of Instruction Execution?
- CPI for a single cycle processor = 1; Multi-cycle processor = k (3,4,or 5)
- CPI for an ideal pipeline(no hazards)?
 - Assume we have n instructions, and k stages
 - The **first instruction** enters the pipeline in cycle 1
 - It leaves the **pipeline** in cycle k
 - The rest of the **(n-1) instructions** leave in the next (n-1) consecutive cycles

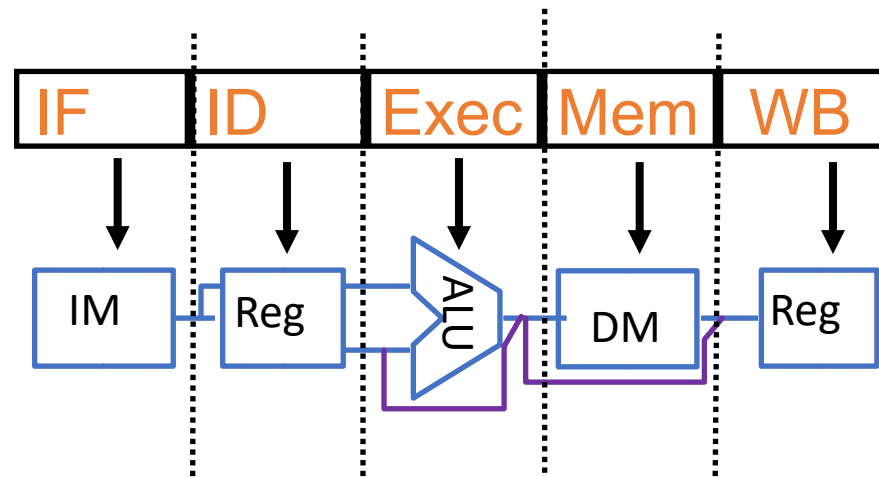
$$CPI_{pipelined} = \frac{n + k - 1}{n}$$

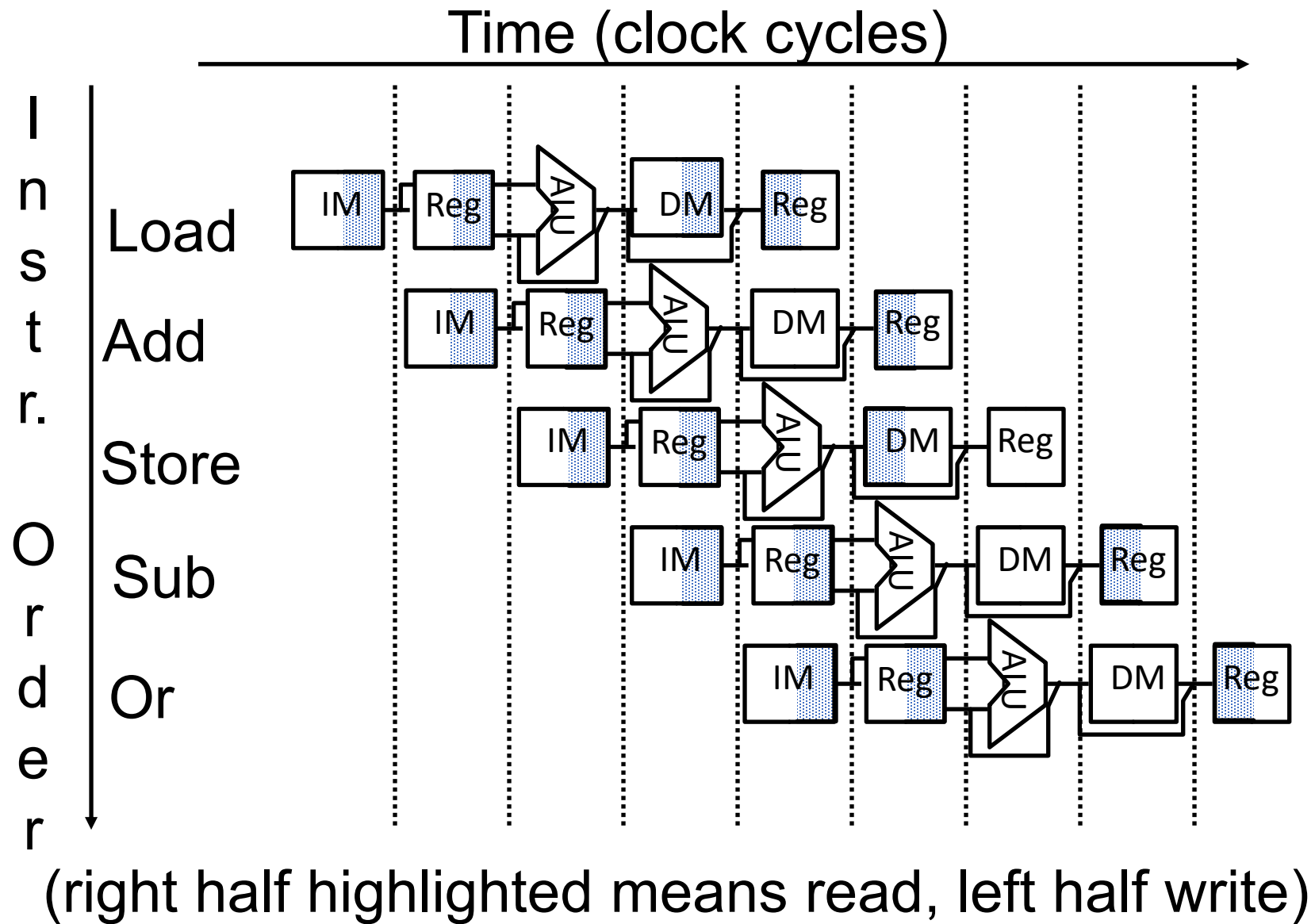
Time per stage $\rightarrow t_{\max} / k$

REVIEW: SINGLE-CYCLE DATAPATH FOR MIPS



- Use datapath figure to represent pipeline



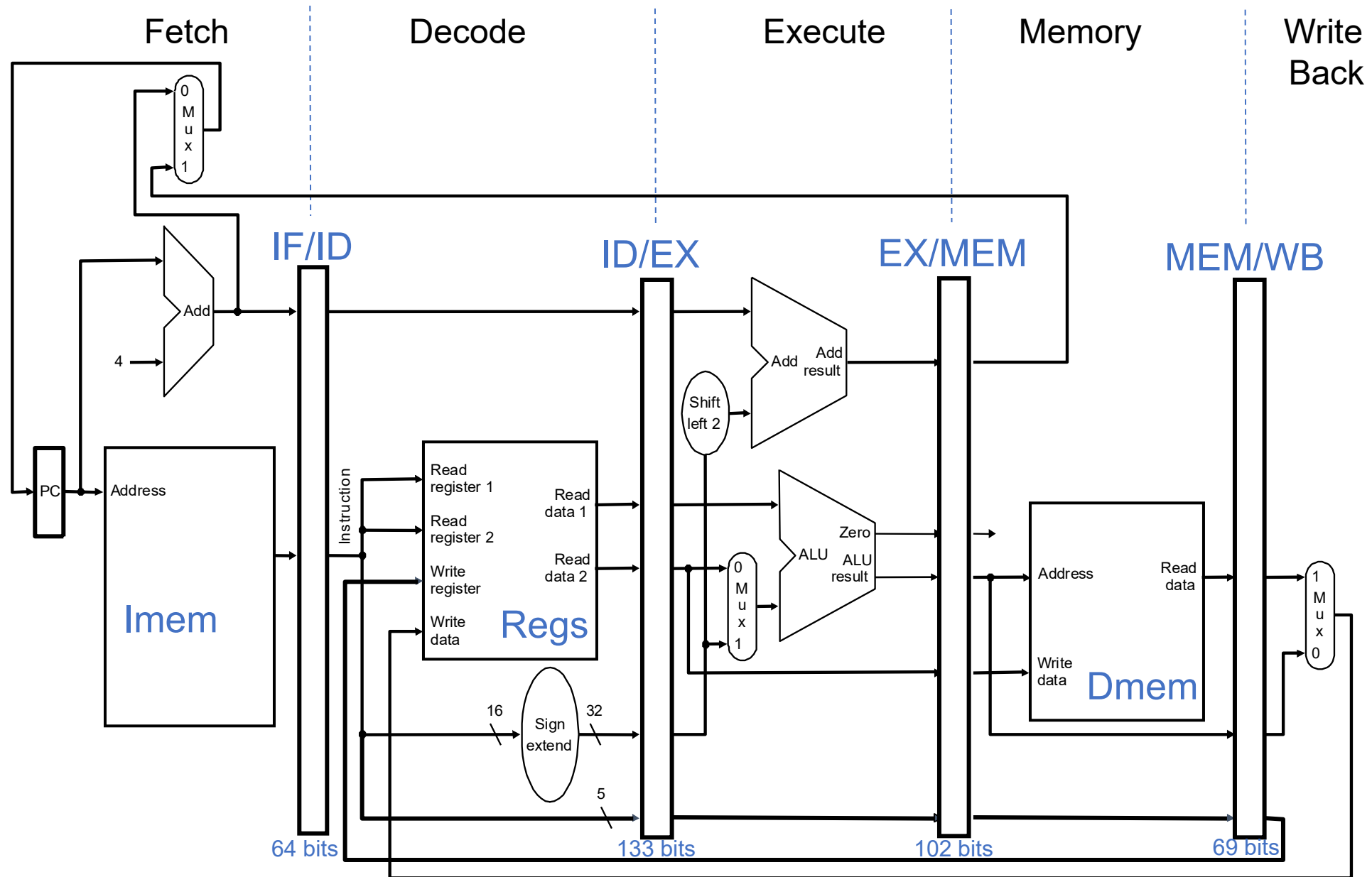


PIPELINED EXECUTION: THE INSTRUCTION PACKET

- What travels **between the pipeline stages** ?
 - The *instruction packet* SAVED IN THE *Pipeline Registers*
- **Instruction Packet (Information necessary to execute each instruction)**
 - i.e. **Instruction contents**
 - **Program counter**
 - **Control signals**
 - **All intermediate results**

Every instruction moves with its entire state; no interference between instructions

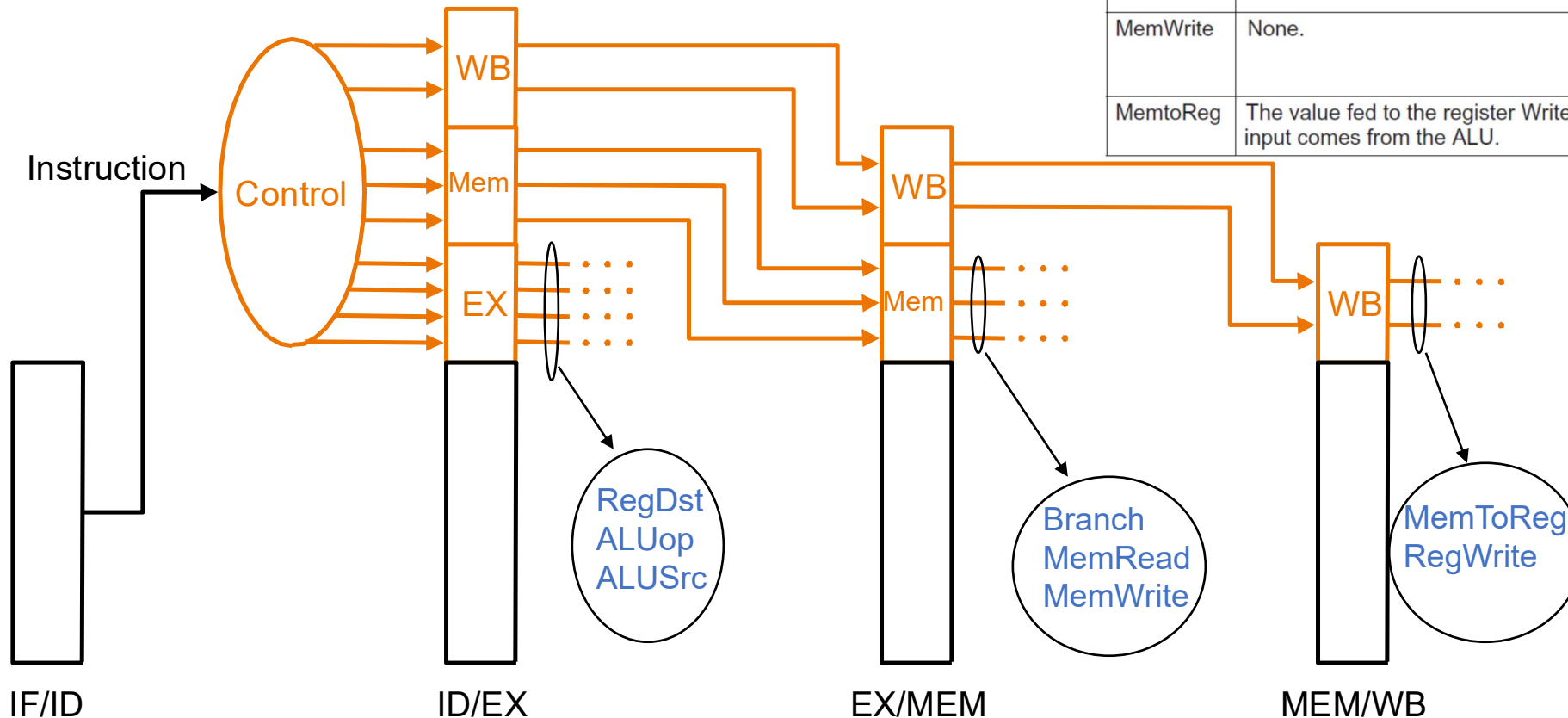
PIPELINED DATAPATH (WITH PIPELINE REGS)



PIPELINED CONTROL

- Start with single-cycle controller
- Group control lines by pipeline stage needed
- Extend pipeline registers with control bits

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.



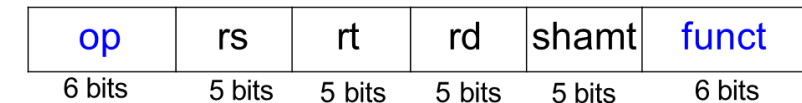
MIPS SUPPORT FOR PIPELINING

- Recall: MIPS was designed to better facilitate pipelining.

Why?

- Same Instruction Length
 - Makes the IF stage universal – easy to prefetch/get to next instruction;
- “Opcode” and SourceRegister* field location always the same
 - Makes possible to read register in ID stage very easily.
- Only Load/Store for memory access
 - Makes the Ex stage to compute the Effective Address.
- Aligned memory access
 - Makes easy and predictable operation at the MA stage.

R-Type



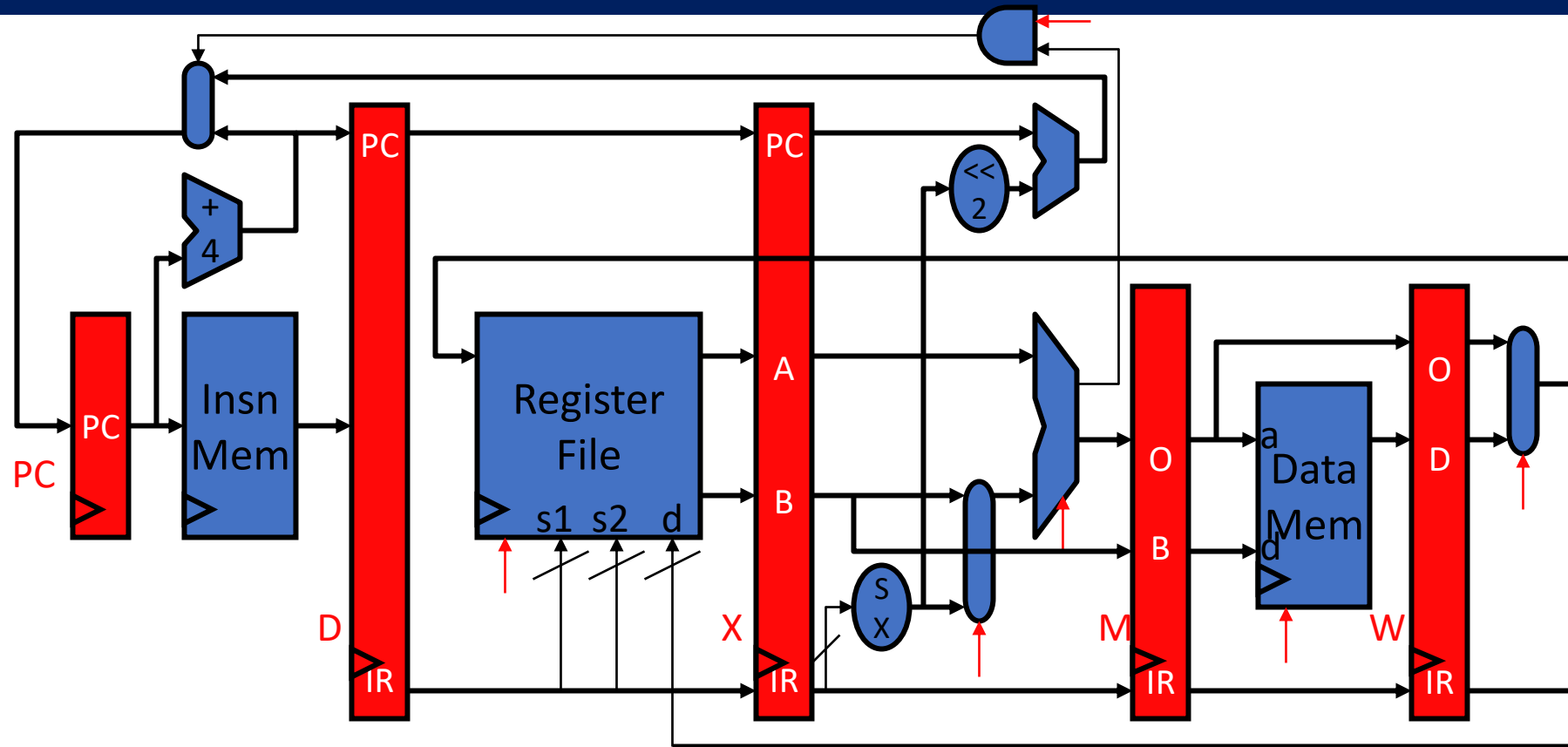
I-Type



J-Type

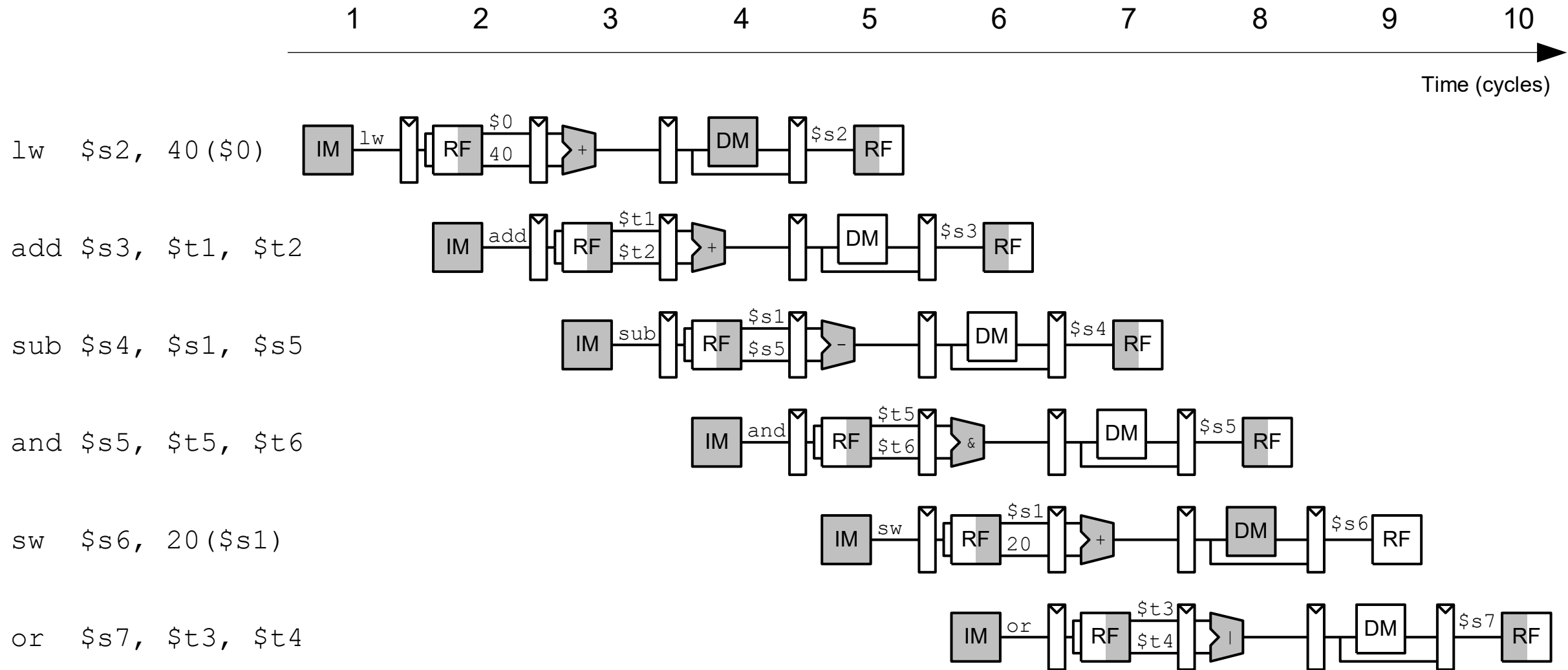


PIPELINE EXAMPLE: CYCLE 0



- Which of the following statements correctly represent the relationship and operations on PC and IR. (32-bit instructions, byte-addressable)
 - IR = PC; PC=PC+4
 - IR = [PC]; PC= PC+4
 - PC = PC+4; IR=PC
 - PC = PC+4; IR=[PC]
 - PC = PC+4; IR = PC

PIPELINED EXECUTION



PIPELINE DIAGRAM

- **Pipeline diagram:** shorthand for what we just saw;

- Across: clock cycles
- Down: instructions
- Cell: <Pipeline Stage>
- assuming no *stalls* (discussed later)

<i>Insn. Stream</i> ↓	<i>Clock Cycles</i> →								
	1	2	3	4	5	6	7	8	9
add \$3,\$2,\$1	F	D	X	M	W				
lw \$4,8(\$5)		F	D	X	M	W			
sw \$6,4(\$7)			F	D	X	M	W		

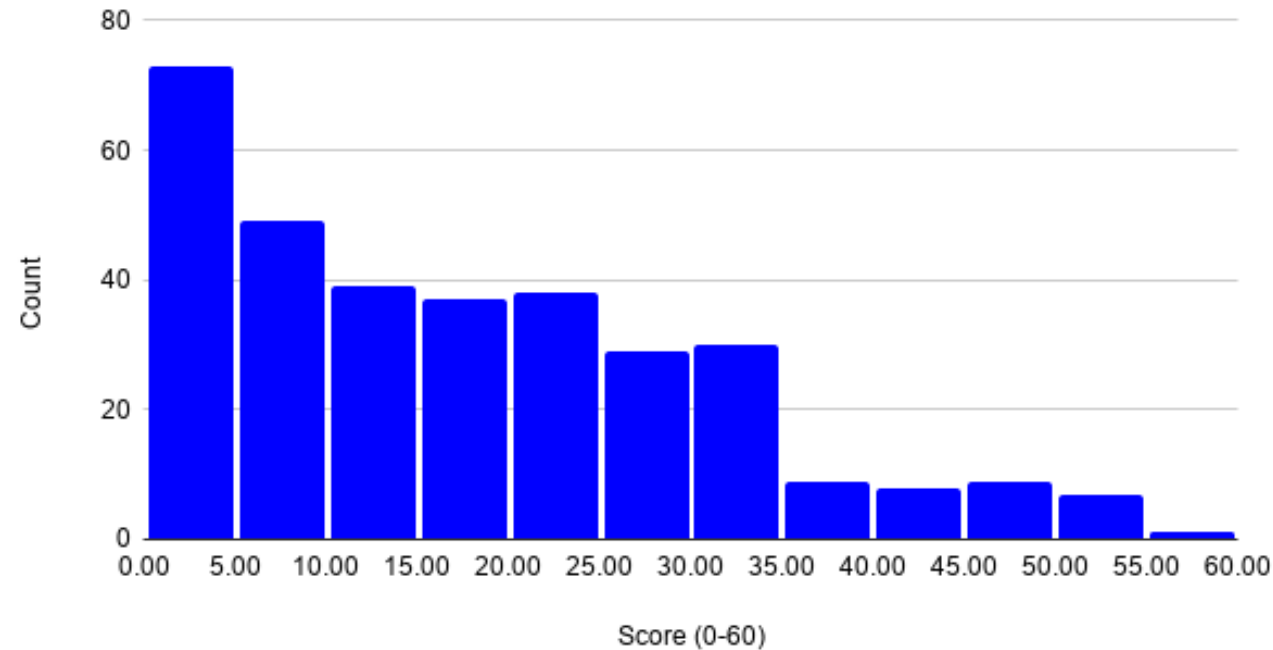
- **Space-Time diagram:** alternate way to express what we just saw;

- Across: clock cycles
- Down: Pipeline Stages
- Cell: <Instruction #>
- assuming no *stalls* (discussed later)

<i>Pipeline Stages</i> ↓	<i>Clock Cycles</i> →								
	1	2	3	4	5	6	7	8	9
Inst. Fetch	I ₁	I ₂	I ₃						
Inst. Decode		I ₁	I ₂	I ₃					
Execution			I ₁	I ₂	I ₃				
Memory Access				I ₁	I ₂	I ₃			
Writeback					I ₁	I ₂	I ₃		

MIDSEM EXAM STATISTICS

Histogram of Midsem Score



Score (0-60)		
STATS	Total (60)	Total (30)
Mean	17.2	8.6
Median	15	7.5
Mode	0	0
Maximum	58	29
Minimum	0	0
# > 50%	77	77

	Q1	Q2 (PSI)	Q3
Mean	4.1	4.7	4.1
Median	4	4	3
Mode	3	0	0
Maximum	14	15	15
Minimum	-6	0	0
# 100%	0	10	1

	Q4 (Prog)	Q5 (Prob)	Q6 (Bon)
Mean	1.1	3.8	0.12
Median	0	0	0
Mode	0	0	0
Maximum	15	15	2.5
Minimum	0	0	0
# 100%	17	40	3

PIPELINE VS. SC AND MC PERFORMANCE

- Assume time for stages is:
 - 100ps for register file read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw						
sw						
R-format						
beq						

PIPELINE VS. SC AND MC PERFORMANCE

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw						
R-format						
beq						

PIPELINE VS. SC AND MC PERFORMANCE

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format						
beq						

PIPELINE VS. SC AND MC PERFORMANCE

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

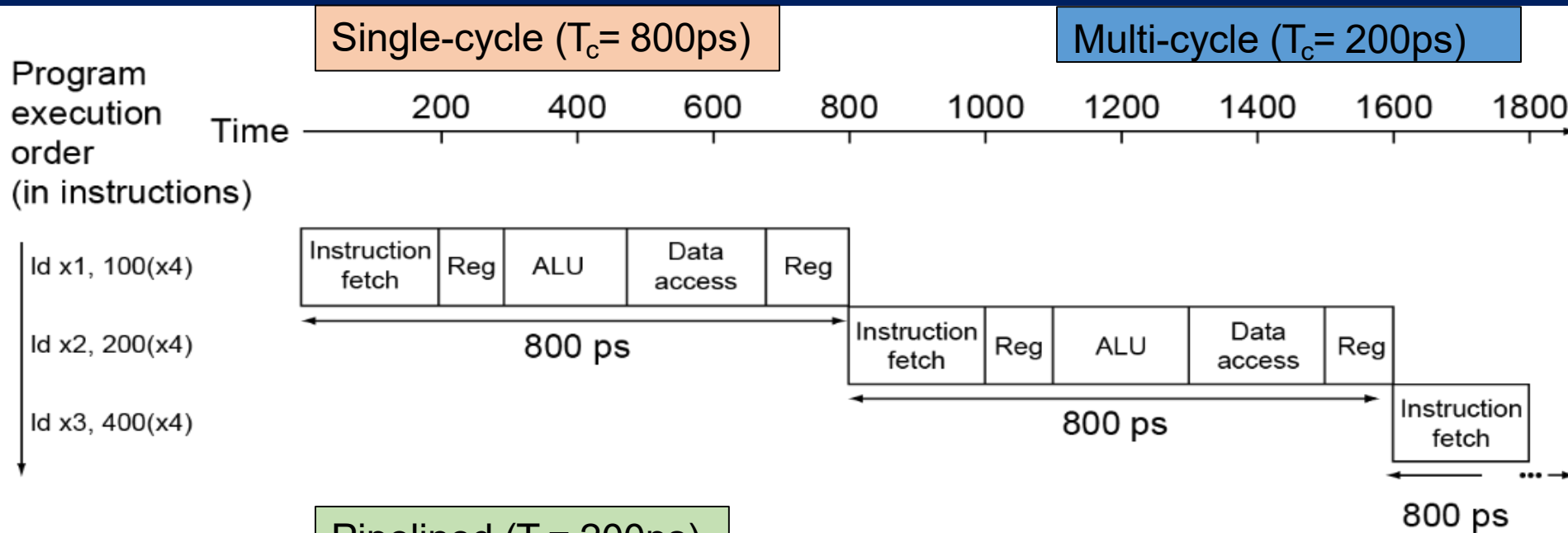
Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq						

PIPELINE VS. SC AND MC PERFORMANCE

- Assume time for stages is
 - 100ps for Register File read or write.
 - 200ps other stages.
- Compare pipelined datapath with single-cycle datapath

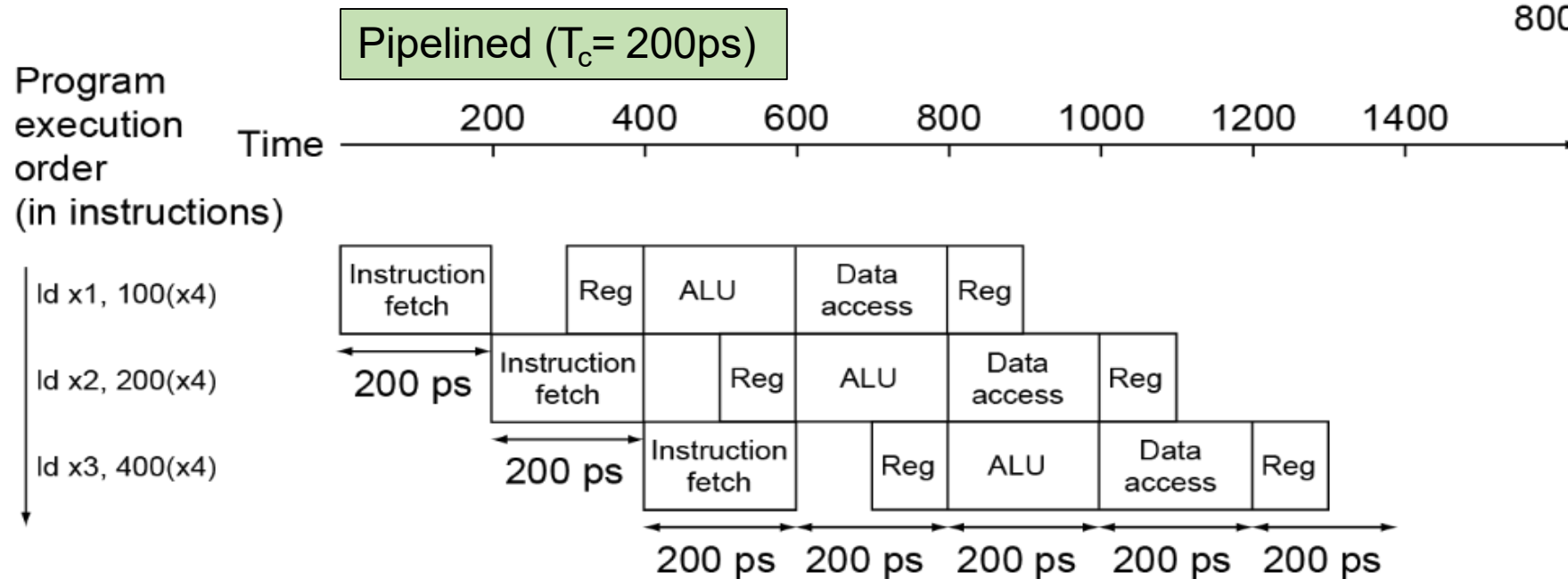
Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

PIPELINE VS. SC AND MC PERFORMANCE



Ex. Time(SC) = $3 \times 800\text{ps}$
= **2400 ps**

Ex. Time(MC) = $3 \times 5 \times 200\text{ps}$
= **3000 ps**



Ex. Time(PP) = $(3 + 5 - 1) \times 200\text{ps}$
= **1400 ps**

Speedup (SC) = **1.71**
Speedup (MC) = **2.14**

- If all stages are balanced
 - i.e., all take the same time

$$\text{Speedup}_{(\text{pipelined})} = \frac{\text{Time between Instructions}_{\text{nonpipelined}}}{\text{Number of pipeline stages}}$$

- If not balanced, speedup would be less.

Note: Speedup is due to increased throughput:

- Latency (time for each instruction) does not decrease
- In fact, latency might increase...

Science is always wrong. It never solves a problem without creating ten more.

George Bernard Shaw

- **Dependence**: relationship between two instructions.
 - **Data**: two instructions use same storage location
 - **Control**: one instruction affects whether another executes or not
 - Not a bad thing, programs would be boring without them
 - Enforced by making older instruction go before younger one
 - Happens naturally in single-/multi-cycle designs
- **Hazard**: dependence & possibility of wrong instruction order
 - Effects of wrong instruction order must not be externally visible
 - **Stall**: for order by keeping younger instruction in same stage
 - Hazards are a bad thing: stalls reduce performance

- Hazards prevent next instruction from executing during its designated clock cycle, thus limiting the speedup
 - Structural hazards: HW cannot support this combination of instructions (single person to fold and put clothes away); *a.k.a. Resource Hazards*.
 - Control hazards: conditional branches & other instructions may stall the pipeline delaying later instructions (must check detergent level before washing next load)
 - Data hazards: Instruction depends on result of prior instruction still in the pipeline (matching socks in later load)

STRUCTURAL HAZARD

- **Structural Hazard**

- When there is a resource conflict or some functional unit is not accessible.

load t0, 0(t1)
add t2, t3, t4
sub t5, t6, t7
and t8, t9, t0

- *Consider Memory unit has **single Read port***

Instr	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
Lw	IF	ID	EX	MEM	WB			
Add	-	IF	ID	EX	MEM	WB		
Sub			IF	ID	EX	MEM	WB	
And				IF	ID	EX	MEM	WB

STRUCTURAL HAZARD – HOW TO RESOLVE

- **Structural Hazard**

- When there is a resource conflict or some functional unit is not accessible.

load t0, 0(t1)
add t2 t3 t4
sub t5 t6 t7
and t8 t9 t0

- Consider Memory unit has **single Read port**
- **Stall the pipeline** → Results in bubbles across stages

Instr	Cycle 1	Cycle2	Cycle 3	Cycle4	Cycle5	Cycle 6	Cycle 7	Cycle 8	Cycle 9
Lw	IF	ID	EX	MEM	WB				
Add	-	IF	ID	EX	MEM	WB			
Sub			IF	ID	EX	MEM	WB		
STALL				Bubble	Bubble	Bubble	Bubble	Bubble	
And				STALL	IF	ID	EX	MEM	WB

- **Flow dependence**

- READ after WRITE (RAW) (*True/Value dependence*)
- Read from a register after a previous instruction wrote into the register

- **Anti Dependence**

- WRITE after READ (WAR)
- Write to a register after a previous instruction read from the register

- **Output dependence**

- WRITE after WRITE (WAW)
- Write to a register after a previous instruction wrote into the register

DATA HAZARD CLASSIFICATION

• Flow dependence --- RAW Hazard

- READ after WRITE (RAW) (*True/Value dependence*)
- [Req] Read from a register after a previous instruction wrote into the register
- RAW Hazard occurs if read by (I_{n+1}) occurs before write by (I_n) – “*read too soon*”

```
I1: lw t0, 100(t1)
I2: add t2, t2, t0
I3: lw t2 100(t1)
I4: or t6 t7 t0
I5: xor t8 t9 t0
```

• Anti Dependence – WAR HAZARD

- WRITE after READ (WAR)
- [Req] Write to a register after a previous instruction read from the register
- WAR Hazard occurs if write by (I_{n+1}) occurs before write by (I_n) – “*write too soon*”

• Output dependence – WAW Hazard

- WRITE after WRITE (WAW)
- [Req] Write to a register after a previous instruction wrote into the register
- WAW Hazard occurs if write by (I_{n+1}) occurs before write by (I_n) – “*written out of order*”

• Data Hazard

- When there is a data dependency i.e. read/write access to operands in specific order.

I1: add t0, t1 t2
 I2: sub t2 t3 t0
 I3: and t4 t5 t0
 I4: or t6 t7 t0
 I5: xor t8 t9 t0

- t0 register is **written in I1** and read by others*
- t0 register will be updated at end of Cycle 5.*
- t0 register will be read by I2, I3, I4 in cycles 2,3,5.*

Instr	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
Add	IF (I1)	ID (I1)	EX (I1)	MEM (I1)	WB (I1)			
Sub	-	IF (I2)	IDs (I2)	EX (I2)	MEM (I2)	WB (I2)		
And			IF (I3)	IDs (I3)	EX (I3)	MEM (I3)	WB (I3)	
OR				IF (I4)	IDs (I4)	EX (I4)	MEM (I4)	WB (I4)
XOR					IF (I5)	ID (I5)	EX (I5)	MEM (I5)

DATA HAZARD

• Data Hazard

- When there is a data dependency i.e. read/write access to operands in specific order.

I1: add **t0**, t1 t2
I2: sub t2 t3 **t0**
I3: and t4 t5 **t0**
I4: or t6 t7 **t0**
I5: xor t8 t9 **t0**

- **T0** register is **written in I1** and read by others
- **T0** register will be updated at end of Cycle 5.
- **T0** register will be read by I2, I3, I4 in cycles 2,3,5.

Instr	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
Add	IF (I1)	ID (I1)	EX (I1)	MEM (I1)	WB (I1)			
sub		IF (I2)	Bubble	Bubble	Bubble			
STALL				Bubble	Bubble	Bubble	Bubble	
STALL				Bubble	Bubble	Bubble	Bubble	Bubble
STALL					Bubble	Bubble	Bubble	Bubble
sub					IF (I3)	ID (I2)	EX (I2)	MEM (I2)

CODE ORDERING (RECAP: INSTRUCTIONS: MORE COMPLEX CODE)

High-level code

```
a = b + c - d;
```

MIPS assembly code

```
add t, b, c # t = b + c
sub a, t, d # a = t - d
```

More complex code is handled by multiple MIPS instructions.

```
a = b + c + d + e;
```

```
add t, b, c # t = b + c
add t, t, d # t = t + d
add a, t, e # a = t + e
```

```
add t, b, c # t = b + c
add t2, d, e # t2 = d + e
add a, t, t2 # a = t + t2
```

- A single line of C code is converted into multiple lines of assembly code

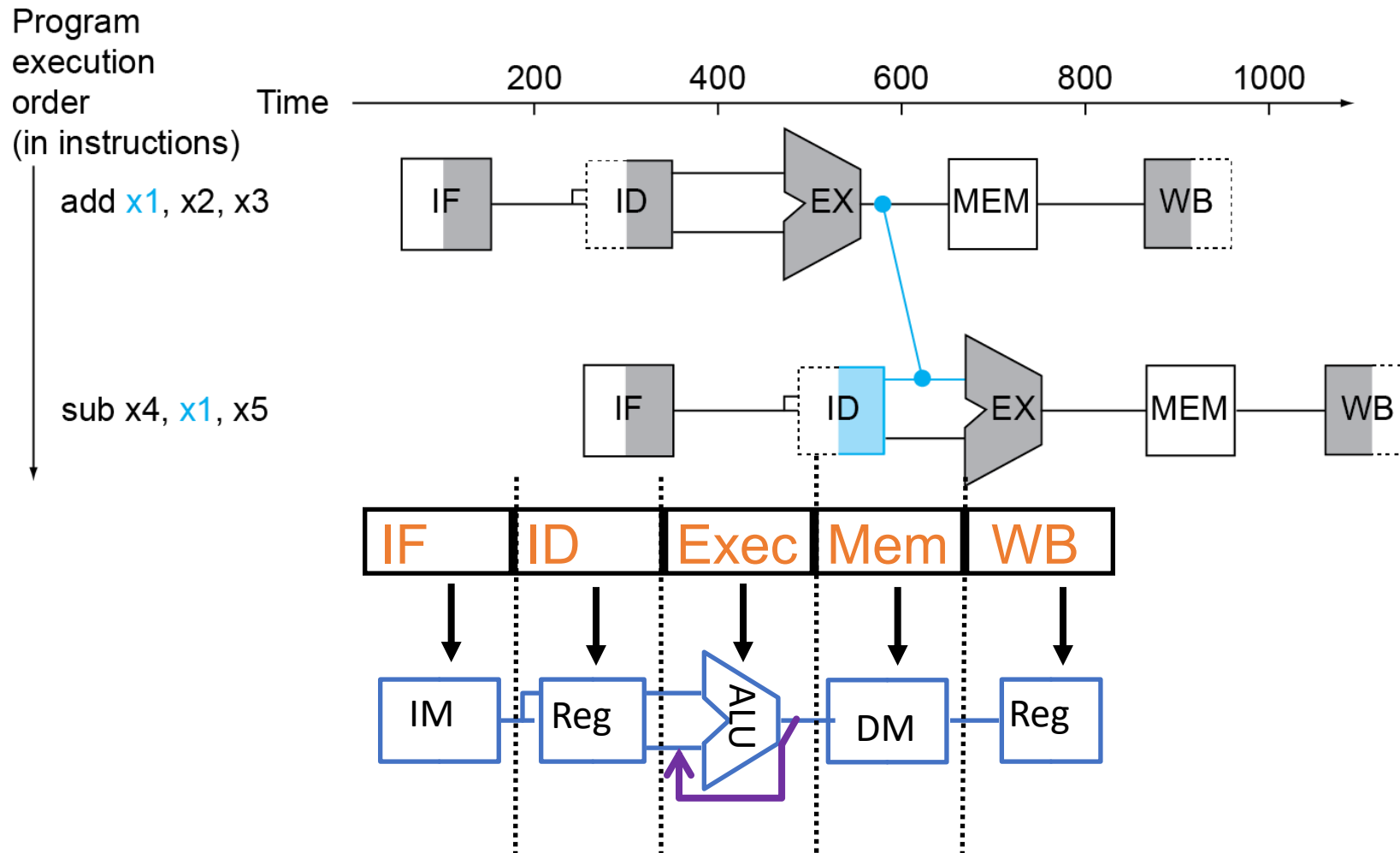
- There can be multiple ways to execute the same HLI

- Some sequences are better than others...
- *the second sequence needs one **more temporary variables** (t2), while it (can perform first two instructions in **parallel**)*

Some sequences are → **better suited for pipelined execution.**

ADDRESSING DATA HAZARDS: FORWARDING (A.K.A. BYPASSING)

- Use result when it is computed
 - Don't wait for it to be stored in a register
 - Requires extra connections in the datapath



RECAP -- PIPELINING HAZARDS

- Hazards prevent next instruction from executing during its designated clock cycle, thus limiting the speedup
- Structural hazards: HW cannot support this combination of instructions (single person to fold and put clothes away)
- Data hazards: Instruction depends on result of prior instruction still in the pipeline (matching socks in later load)
- Control hazards: conditional branches & other instructions may stall the pipeline.-- delaying later instructions (must check detergent level before washing next load)